

ForgeCloud Docker & Kubernetes

Containerization, EKS deployment, health checks and autoscaling

1. Container Strategy

Docker packages applications consistently so that the same application artifact can move from local development to the Kubernetes runtime.

2. Docker Components

```
frontend/  
  Dockerfile  
  
backend/  
  Dockerfile  
  
docker-compose.yml  
  
Local:  
PostgreSQL + FastAPI + React
```

3. Kubernetes Architecture

```
EKS Cluster  
|  
+-- Namespace  
|  
+-- Deployment  
|   +-- Pod  
|   +-- Pod  
|   +-- Pod  
|  
+-- Service  
|  
+-- Ingress  
|  
+-- ConfigMap  
|  
+-- Secret references  
|  
+-- HPA  
|  
+-- Health probes  
|  
+-- Resource requests/limits
```

4. Deployment Configuration

- Container image from Amazon ECR.
- Kubernetes Deployment controls desired replicas.
- Service provides stable service discovery.
- Ingress exposes HTTP/HTTPS traffic.
- ConfigMaps hold non-secret configuration.
- Secrets are referenced without committing credentials.
- Readiness probes control whether a pod receives traffic.
- Liveness probes detect unhealthy containers.

- Resource requests and limits support predictable scheduling.
- HPA changes replicas based on resource utilization.

5. Autoscaling Flow

```
Traffic increases
|
CPU / memory utilization increases
|
HPA observes utilization
|
Replica count increases
|
Application capacity increases
|
Utilization returns toward target
```

6. Failure Recovery

```
Pod failure
-> Kubernetes detects failure
-> replacement pod created
-> readiness probe passes
-> service routes traffic
-> desired replica count restored
```